Project Title

Plagiarism Detection Engine for Academic Submissions

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Erigela Pranay Reddy (2520080061)
Palavalasa Rahul (2520090016)

Team Number: 4
Section: 11

Under the guidance of

Mrs. Ch. Anitha

Asst. Professor, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled "**Plagiarism Detection Engine for Academic Submissions**" is a record of Bonafide work of **Erigela Pranay Reddy – 2520080061.**, **Palavalasa Rahul – 2520090016.** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Erigela Pranay Reddy – 2520080061.

Palavalasa Rahul – 2520090016.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the mini project based report entitled "**Plagiarism Detection Engine for Academic Submissions**" is a bonafide work done and submitted by **Erigela Pranay Reddy – 2520080061., Palavalasa Rahul – 2520090016.** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Mrs. Ch. Anitha, Asst. Professor, CS&IT

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our guide, **Mrs. Ch. Anitha**, Asst. Professor, CS&IT, Department of Computer Science and Information Technology, for her tremendous support, encouragement and guidance throughout this project, which cannot be expressed in words.

We express our gratitude to **Dr. Khaja Shareef (CSIT)**, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff, and all those who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

Erigela Pranay Reddy – 2520080061.

Palavalasa Rahul – 2520090016.

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	17
8.	Conclusion & Future Scope	21
9.	References	22

1. ABSTRACT

The Plagiarism Detection Engine for Academic Submissions is an intelligent system developed to detect and analyze similarities in student assignments with high accuracy and efficiency. The project aims to assist educational institutions in maintaining academic integrity by identifying copied or near-duplicate content while minimizing false detections. Initially, the system indexes a corpus of student submissions and applies Rabin–Karp fingerprinting along with the Winnowing algorithm to generate compact document fingerprints and efficiently shortlist candidate document pairs. For each shortlisted pair, a Suffix Array is constructed, and the Longest Common Prefix (LCP) Array is used to identify maximal common substrings that exceed a predefined similarity threshold. The detected matching sections are highlighted in a side-by-side comparison report, enabling users to easily examine overlapping content. The system also calculates an overall similarity score and performs false-positive analysis by comparing results against a hand-labeled dataset to evaluate detection accuracy. By combining efficient indexing techniques with advanced string-matching algorithms, the proposed solution offers a scalable, reliable, and computationally efficient plagiarism detection framework suitable for universities, colleges, and online learning platforms handling large volumes of academic submissions every semester.

Team Members:

1. Erigela Pranay Reddy, 2520080061
2. Palavalasa Rahul, 2520090016

2. INTRODUCTION

2.1 Background

Plagiarism is the use of another person's work or ideas, presented as one's own, without proper attribution. In academic settings it undermines fair evaluation, discourages genuine learning and damages the credibility of the institution. As coursework is now submitted digitally, one instructor may receive hundreds of files in a semester, and text can be copied with a few keystrokes.

Comparing every submission against every other submission by hand is slow, inconsistent and practically impossible at this scale. Simple word-by-word comparison is also easy to defeat: a student only has to reorder sentences or replace a few words to hide a copied passage. A dependable solution needs algorithms that are fast over large collections and precise about where two documents overlap.

2.2 Motivation

Hashing reduces long passages to small numbers that can be compared quickly, and suffix-based structures can find shared text without scanning every pair of positions. The project is motivated by the chance to combine these classical techniques in one working system and to see how their complexity trade-offs behave on real text.

2.3 What, Why and How

- **What:** a plagiarism detection engine that indexes a corpus of student submissions, shortlists suspicious document pairs, locates the exact matching passages, scores their similarity and presents them in a highlighted side-by-side report.
- **Why:** manual checking does not scale, naive comparison is slow (n documents require $n(n-1)/2$ pair comparisons) and aggressive detection produces false positives that unfairly accuse honest students.
- **How:** Rabin–Karp rolling hashes and the Winnowing algorithm create compact fingerprints and an inverted index that shortlists candidate pairs. A Suffix Array with an LCP Array then finds maximal common substrings in each shortlisted pair, and the results are validated against a hand-labelled dataset.

2.4 Organisation of the Report

Chapter 3 surveys existing systems, Chapter 4 states the problem and objectives, Chapter 5 lists the requirements, and Chapter 6 describes the design and the module code. Chapter 7 presents the results and false-positive analysis, Chapter 8 concludes with future scope, and Chapter 9 lists the references.

3. LITERATURE SURVEY

This chapter reviews earlier systems and research that address document and source-code similarity. Each entry summarises the approach, and the full references with links are given in Chapter 9.

3.1 MOSS (Measure of Software Similarity)

MOSS is a widely used similarity service for programming assignments, developed at Stanford University. It is built on the Winnowing algorithm, which selects a small, guaranteed subset of k-gram hashes from each file so that comparisons stay cheap. MOSS ignores whitespace and renaming effects and reports the matching regions of each pair of files. It is highly effective for source code, but it is a hosted service and is not designed to explain matches through suffix-based analysis. [5]

3.2 JPlag

JPlag finds similarities among sets of programs by converting each program into a stream of language tokens and then comparing the token streams with the Greedy String Tiling algorithm. Working on tokens makes it robust against renamed variables and changed formatting. The idea of finding and merging long common pieces of two sequences is close to the maximal common substring step used in this project, although JPlag does not use fingerprint indexing for shortlisting. [6]

3.3 Turnitin

Turnitin is a commercial similarity-checking service widely used by universities. A submitted document is compared against a very large repository of web pages, journals and previously submitted student papers, and an originality report with highlighted passages is produced. Its strength is the size of its repository; its limitations are cost, dependence on an external database and limited transparency about the exact matching method. [11]

3.4 COPS and SCAM Copy Detection

Brin, Davis and Garcia-Molina proposed copy-detection mechanisms for digital documents, in which registered documents are broken into sentence or word chunks and stored in a database so that new documents can be checked against them. The work showed that chunk-level indexing is

practical for large collections. It is the origin of the “index first, compare later” strategy that our fingerprint index follows. [7]

3.5 Shingling and Resemblance (Broder)

Broder introduced w-shingling, where a document is treated as the set of its contiguous word sequences, and the resemblance of two documents is estimated from a random sample of shingle hashes. This makes near-duplicate detection feasible for web-scale collections. The approach estimates how similar two documents are but does not locate the exact overlapping passages, which our suffix-array stage adds. [8]

3.6 Sherlock

Sherlock, developed by Joy and Luck, compares student programs by removing comments, whitespace and identifier differences and then searching for common sequences of lines. It was designed for use in computing courses and demonstrated that automatic similarity detection can support fair marking at departmental level. Its pairwise comparison of all submissions grows quadratically, which is the drawback that fingerprint shortlisting is meant to reduce. [9]

3.7 Dolos

Dolos is an open-source plagiarism detection tool for source code. It tokenises submissions, builds k-gram fingerprints, applies winnowing to select representative fingerprints, and presents an interactive visualisation of the matching fragments. It shows the value of a clear visual report for teachers, which motivates the side-by-side highlighted report produced by our engine. [10]

3.8 Summary of the Gap

Existing tools are strong at either large-scale fingerprint comparison (MOSS, Broder, Dolos) or detailed matching (JPlag, Sherlock). The proposed engine combines both stages in one pipeline: Rabin–Karp and Winnowing reduce the number of pairs that must be compared, and a Suffix Array with an LCP Array then gives exact, maximal matching passages that can be highlighted and explained. A false-positive analysis against a hand-labelled dataset is built in, so the reliability of the similarity threshold can be measured instead of assumed.

4. PROBLEM STATEMENT AND OBJECTIVES

4.1 Problem Statement

To design an efficient plagiarism detection engine that identifies copied and near-duplicate passages in a large corpus of student submissions while reducing false positives and producing interpretable similarity reports.

4.2 Problem Description

Comparing every pair of submissions character by character is too slow for large classes, and simple matching either misses edited copies or flags harmless common phrases. The system must shortlist likely pairs quickly, find the exact overlapping text, and let evaluators see and trust the evidence.

4.3 Objectives

The project was initiated to support academic integrity by giving evaluators a fast and explainable way to find copied work. The objectives derived from this purpose are:

1. To index student submissions efficiently.
2. To generate compact document fingerprints using Rabin–Karp hashing and the Winnowing algorithm.
3. To shortlist potentially similar document pairs and avoid unnecessary comparisons.
4. To detect maximal common substrings using a Suffix Array and an LCP Array.
5. To calculate an overall similarity score for each compared pair.
6. To highlight the matching passages in a side-by-side comparison report.
7. To analyse false positives against a hand-labelled dataset.
8. To produce an understandable plagiarism report for evaluators.

5. HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Requirements

- Computer / laptop with a multi-core processor
- Minimum 4 GB RAM
- Adequate storage for the document corpus and generated reports

5.2 Software Requirements

- Operating system: Windows or Linux
- Programming language: Python 3 (standard library modules re, collections and itertools are used by the engine)
- Code editor / IDE such as Visual Studio Code
- Matplotlib for generating the comparison report, similarity matrix and graphs
- Dataset of student submissions in plain-text form, with a hand-labelled list of plagiarised pairs for evaluation

6. IMPLEMENTATION

6.1 System Workflow

The engine processes a batch of submissions in the following twelve steps:

1. Student documents are loaded.
2. Text preprocessing converts each document to lower case and removes punctuation and extra spaces.
3. Rabin–Karp fingerprinting computes a rolling hash for every k-gram.
4. The Winnowing algorithm selects representative fingerprints from the hashes.
5. Candidate pair selection uses an inverted index to find documents that share enough fingerprints.
6. A Suffix Array is constructed for each candidate pair.
7. The LCP Array is computed from the Suffix Array.
8. Common substring detection reads maximal matches from adjacent suffixes that belong to different documents.
9. A similarity threshold removes very short matches.
10. The similarity score is calculated from the matched text.
11. A highlighted side-by-side plagiarism report is generated.
12. False-positive analysis compares the flagged pairs with the hand-labelled dataset.

6.2 System Architecture

The system is organised in five layers. The **Input Layer** accepts the student submissions. The **Processing Layer** performs preprocessing, fingerprinting and winnowing. The **Detection Layer** selects candidate pairs and runs the Suffix Array and LCP analysis. The **Analysis Layer** calculates similarity, checks thresholds and carries out false-positive analysis. The **Output Layer** produces the side-by-side comparison, the highlighted passages and the similarity report. Because only shortlisted pairs reach the expensive Detection Layer, the design scales to large numbers of submissions.

6.3 Algorithms and Parameters

- **Rabin–Karp rolling hash:** the hash of the next k-gram is computed from the previous one in constant time by removing the leftmost character and adding the new character. Parameters: k = 8 characters, base 257, modulus 2^61 − 1.
- **Winnowing:** in each window of w = 4 consecutive hashes the smallest hash (rightmost on ties) is selected. Every shared passage of at least w + k − 1 = 11 characters is guaranteed to share at least one selected fingerprint.
- **Candidate selection:** an inverted index maps each fingerprint to the documents that contain it. A pair is shortlisted when it shares at least 10 fingerprints.
- **Suffix Array and LCP Array:** the two documents are joined with separator characters and all suffixes are sorted. Adjacent suffixes that come from different documents and share a prefix of at least 25 characters mark a common substring; matches that can be extended to the left are discarded so that only maximal matches remain.
- **Similarity score:** (matched characters in document A + matched characters in document B) ÷ (length of A + length of B), after overlapping matches are merged. A pair is flagged when the score is at least 30%.

The suffix array of the word “banana” illustrates how the LCP values expose shared text. Neighbouring suffixes that start with “ana” share three characters, which is the signal used to find matching passages.

Rank	Start index	Suffix	LCP
0	5	`a`	0
1	3	`ana`	1
2	1	`anana`	3
3	0	`banana`	0
4	4	`na`	0
5	2	`nana`	2

Table 6.1: Sorted suffixes and LCP values for “banana”

6.4 Complexity

- Preprocessing and Rabin–Karp hashing: O(n) per document of length n.
- Winnowing: O(n·w) in this implementation, which is linear for a fixed window size.

- Suffix Array construction by prefix doubling with sorting: $O(n \log^2 n)$; LCP Array (Kasai's algorithm): $O(n)$.
- Merging matched intervals: $O(m \log m)$ for m matches.

Using the inverted index, the expensive suffix-array stage is applied only to shortlisted pairs instead of all $n(n-1)/2$ pairs.

6.5 Module Code

The engine is written in Python. The source code of the four main modules is given below.

Module 1: Preprocessing, Rabin–Karp Hashing and Winnowing

```
import re
from collections import defaultdict
from itertools import combinations


K, W = 8, 4                         # k-gram length, winnowing window
BASE, MOD = 257, (1 << 61) - 1      # Rabin-Karp parameters
MIN MATCH = 25                      # minimum common-substring length (chars)


def preprocess(text):
    text = re.sub(r"[^a-z0-9 ]+", " ", text.lower())
    return re.sub(r"\s+", " ", text).strip()


def rabin karp hashes(s, k=K):
    if len(s) < k:
        return []
    h, top = 0, pow(BASE, k - 1, MOD)
    out = []
    for i, ch in enumerate(s):
        if i >= k:
            h = (h - ord(s[i - k]) * top) % MOD      # drop leftmost char
        h = (h * BASE + ord(ch)) % MOD               # add new char
        if i >= k - 1:
            out.append((h, i - k + 1))
    return out


def winnow(hashes, w=W):
    fp, last = set(), None
    for i in range(len(hashes) - w + 1):
        window = hashes[i:i + w]
        m = min(h for h,  in window)
        pos = max(j for j in range(w) if window[j][0] == m)   # rightmost min
        sel = (window[pos][0], window[pos][1])
        if sel != last:
            fp.add(sel)
            last = sel
    return fp
```

Module 2: Indexing and Candidate Pair Selection

```
def build_index(docs):
    index, prints = defaultdict(set), {}
    for name, raw in docs.items():
        s = preprocess(raw)
        prints[name] = {h for h, _ in winnow(rabin_karp_hashes(s))}
        for h in prints[name]:
            index[h].add(name)
    return index, prints
```

```
def candidate pairs(index, prints, min shared=10):
    shared = defaultdict(int)
    for h, owners in index.items():
        for a, b in combinations(sorted(owners), 2):
            shared[(a, b)] += 1
    return {p: c for p, c in shared.items() if c >= min_shared}
```

Module 3: Suffix Array, LCP Array and Common Substring Detection

```
def suffix_array(s):
    n = len(s)
    rank = [ord(c) for c in s]
    sa, k = list(range(n)), 1
    while True:
        key = lambda i: (rank[i], rank[i + k] if i + k < n else -1)
        sa.sort(key=key)
        tmp = [0] * n
        for i in range(1, n):
            tmp[sa[i]] = tmp[sa[i - 1]] + (key(sa[i - 1]) < key(sa[i]))
        rank = tmp
        if rank[sa[-1]] == n - 1:
            break
        k <<= 1
    return sa

def lcp array(s, sa):                       # Kasai, O(n)
    n = len(s)
    rank, lcp, h = [0] * n, [0] * n, 0
    for i, p in enumerate(sa):
        rank[p] = i
    for i in range(n):
        if rank[i] > 0:
            j = sa[rank[i] - 1]
            while i + h < n and j + h < n and s[i + h] == s[j + h]:
                h += 1
            lcp[rank[i]] = h
            h = max(h - 1, 0)
        else:
            h = 0
    return lcp

def common_substrings(a, b, min_len=MIN_MATCH):
    s = a + "\x01" + b + "\x02"
    sa = suffix_array(s)
    lcp = lcp array(s, sa)
    la, out = len(a), []
    for r in range(1, len(sa)):
        p, q = sa[r - 1], sa[r]
        if lcp[r] < min_len or (p < la) == (q < la):
            continue
        pa, pb = (p, q - la - 1) if p < la else (q, p - la - 1)
        if pa > 0 and pb > 0 and a[pa - 1] == b[pb - 1]:
            continue                        # not left-maximal
        out.append((pa, pb, lcp[r]))
    return out

def merge(intervals):
    res = []
    for s, e in sorted(intervals):
        if res and s <= res[-1][1]:
            res[-1][1] = max(res[-1][1], e)
        else:
            res.append([s, e])
    return res
```

Module 4: Similarity Scoring, Report Analysis and Evaluation Driver

```
def compare(a raw, b raw, min len=MIN MATCH):
    a, b = preprocess(a_raw), preprocess(b_raw)
```

```
    ms = common_substrings(a, b, min_len)
    ia = merge([(p, p + l) for p, _, l in ms])
    ib = merge([(q, q + l) for _, q, l in ms])
    cov_a = sum(e - s for s, e in ia)
    cov_b = sum(e - s for s, e in ib)
    score = (cov_a + cov_b) / (len(a) + len(b)) if a and b else 0.0
    return dict(a=a, b=b, spans_a=ia, spans_b=ib, matches=ms, score=score)


def analyse(docs, threshold=0.30):
    index, prints = build_index(docs)
    cands = candidate_pairs(index, prints)
    results = {p: compare(docs[p[0]], docs[p[1]]) for p in cands}
    flagged = {p for p, r in results.items() if r["score"] >= threshold}
    return cands, results, flagged


# Driver (run.py): evaluate against the hand-labelled pairs
from itertools import combinations
from engine import analyse
from corpus import D, LABELS

all_pairs = list(combinations(sorted(D), 2))
cands, results, flagged = analyse(D, threshold=0.30)
tp = len(flagged & LABELS); fp = len(flagged - LABELS)
fn = len(LABELS - flagged); tn = len(all_pairs) - tp - fp - fn
```

7. RESULTS & DISCUSSION

The engine was tested on a sample corpus of 10 short text documents prepared for testing (D01 to D10). The corpus contains original documents on different topics, one exact copy with small edits, one copy with reordered and rewritten sentences, and one partial copy. Two unrelated documents deliberately share a common definition sentence so that the false-positive behaviour can be observed. A hand-labelled list marks the plagiarised pairs: D01–D02, D01–D03, D02–D03 and D04–D05. Parameters: k = 8, w = 4, minimum shared fingerprints = 10, minimum match length = 25 characters, flag threshold = 30%.

7.1 Program Output

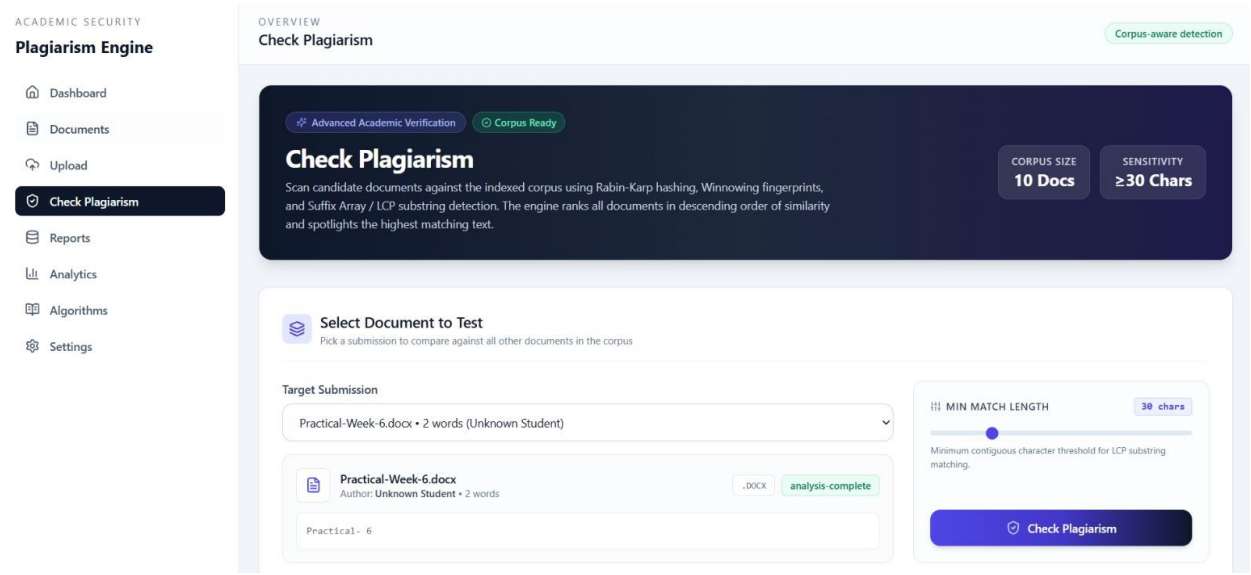


Figure 7.1: Console output of the engine showing shortlisted pairs, similarity scores and evaluation counts

Of the 45 possible pairs, only 7 were shortlisted by the fingerprint index. Four pairs reached the 30% threshold and were flagged.

7.2 Side-by-Side Highlighted Reports

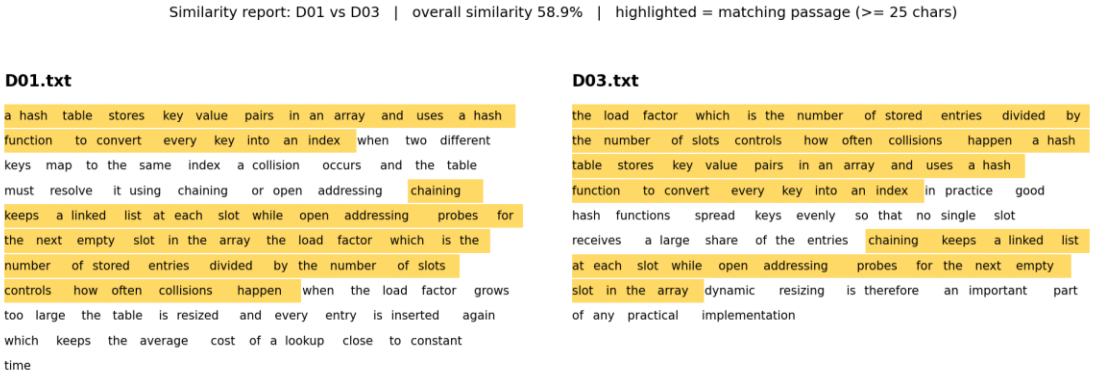


Figure 7.2: Report for D01 and D03 (similarity 58.9%)

D03 reuses the sentences of D01 in a different order. The suffix array locates each reused sentence separately, so the reordered copy is still detected and every matching passage is highlighted in both documents.

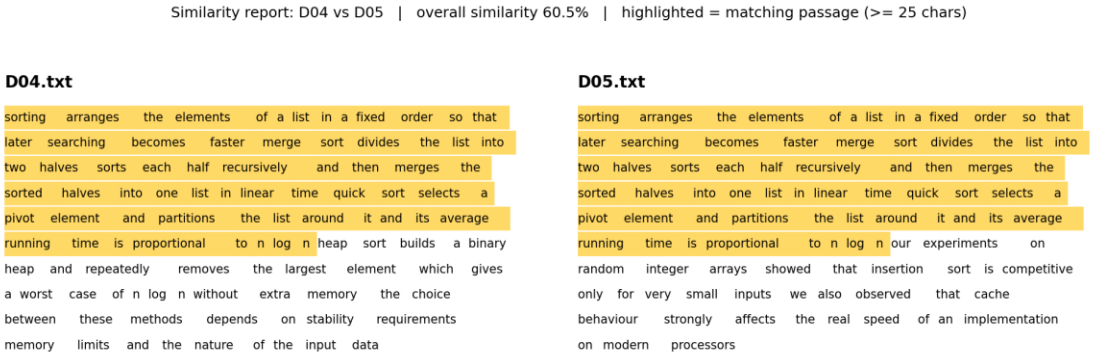


Figure 7.3: Report for D04 and D05 (similarity 60.5%)

D05 copies the first three sentences of D04 and then continues with new text. Only the copied opening is highlighted, which shows that partial plagiarism is reported accurately rather than as a whole-document match.

7.3 Similarity Matrix

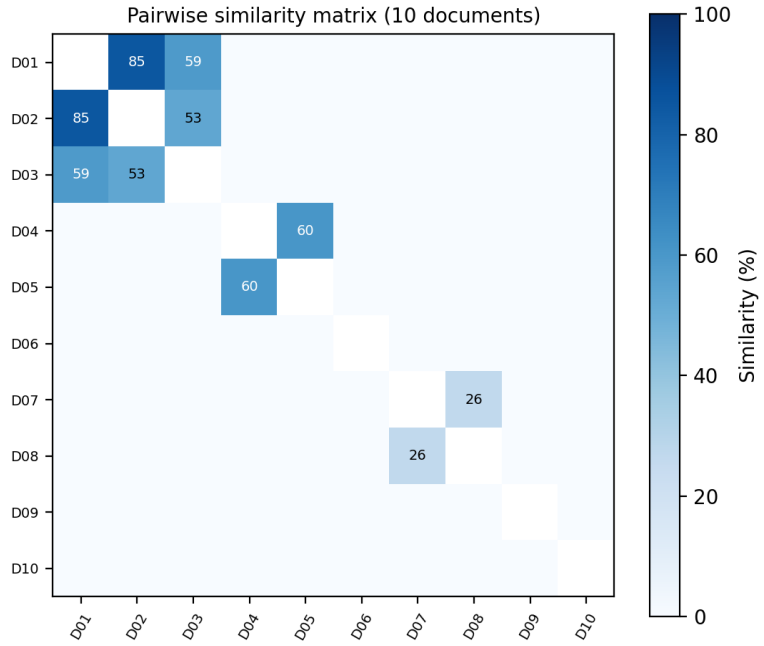


Figure 7.4: Pairwise similarity scores for all documents

The exact copy D01–D02 scores highest at 84.8%. The other plagiarised pairs score between 53.3% and 60.5%, while unrelated pairs score 0%.

7.4 False-Positive Analysis

Similarity report: D07 vs D08 | overall similarity 26.3% | highlighted = matching passage (>= 25 chars)

D07.txt

computer networks connect devices so that they can exchange data over wired or wireless links the layered model separates responsibilities with the transport layer providing reliable delivery between processes and the network layer routing packets between hosts routers forward packets by consulting a table of destination prefixes and congestion control prevents senders from overwhelming the link s a protocol is a set of agreed rules that defines the format and the order of the messages exchanged between two communicating parties

D08.txt

an operating system manages the hardware of a computer and offers services to application programs scheduling decides which process receives the processor next and virtual memory lets every process believe that it owns a large private address space file systems organise persistent data on disks and device drivers hide the differences between hardware device s a protocol is a set of agreed rules that defines the format and the order of the messages exchanged between two communicating parties

Figure 7.5: Report for D07 and D08, an unrelated pair that shares one common definition (26.3%)

D07 and D08 are not plagiarised, but both contain the same standard definition of a protocol, and the pair shared 55 fingerprints. Its score of 26.3% stays below the 30% threshold, so it is not flagged. D04–D10 and D05–D10 were also shortlisted by the index but scored 0% after the suffix-array stage found no common passage of 25 characters, showing that the second stage removes false candidates from the first.

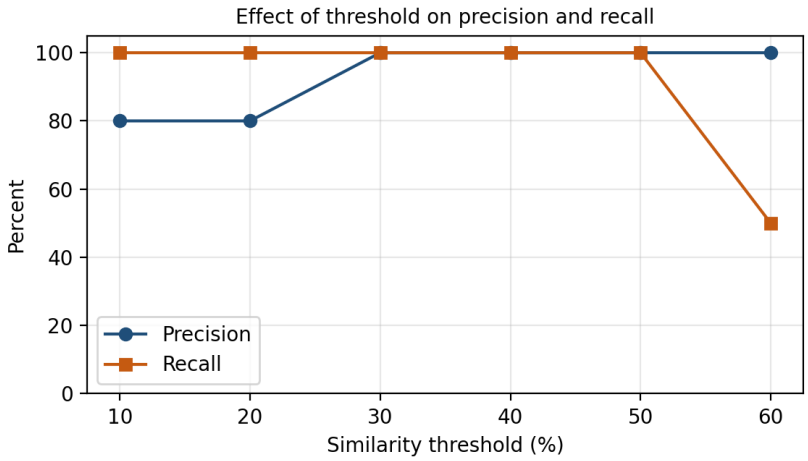


Figure 7.6: Precision and recall for different similarity thresholds

At the 30% threshold the engine produced 4 true positives, 0 false positives, 0 false negatives and 41 true negatives, giving 100% precision and 100% recall on this corpus. Lowering the threshold to 20% or below flags D07–D08 and precision falls to 80%, whereas raising it to 60% misses two plagiarised pairs and recall falls to 50%. A threshold between 30% and 50% was therefore suitable here.

7.5 Efficiency

The fingerprint index reduced the suffix-array comparisons from 45 pairs to 7 pairs, a reduction of 84.4%. On the test machine, comparing every pair took about 0.13 s against about 0.03 s for the shortlisted pairs. The timings depend on the machine, but the saving grows with the number of documents because the number of pairs grows quadratically.

7.6 Discussion

The results show that the two-stage design works as intended: fingerprints give a cheap first filter, and the suffix array gives exact evidence. The reported figures come from a small test corpus, so they demonstrate the behaviour of the method rather than its accuracy on real classroom data. The engine works on character-level matches and therefore cannot detect passages that are paraphrased with different wording.

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

The project set out to build an efficient plagiarism detection engine for academic submissions, and all of its objectives were met. Submissions are indexed through Rabin–Karp and Winnowing fingerprints, candidate pairs are shortlisted, and a Suffix Array with an LCP Array finds maximal common passages that are scored and highlighted in a side-by-side report. On the test corpus the engine detected exact, reordered and partial copies, avoided flagging a pair that shared only a standard definition, and reduced the number of detailed comparisons by more than 80%. The combination of efficient indexing with exact string matching gives a scalable and explainable framework that is suitable for colleges, universities and online learning platforms.

8.2 Future Scope

Near term

- Support for multiple document formats such as PDF, DOCX and source code.
- Evaluation on larger, real institutional document collections.
- Advanced analytics dashboards for evaluators.

Medium term

- Improved semantic and synonym-based detection to catch paraphrased text.
- Machine-learning-assisted classification of flagged pairs.
- Cloud-based deployment and real-time plagiarism checking at submission time.

Long term

- Cross-language plagiarism detection.
- Integration with Learning Management Systems (LMS) for automatic checking of every submission.

9. REFERENCES

1. S. Schleimer, D. S. Wilkerson and A. Aiken, “Winnowing: Local algorithms for document fingerprinting,” in Proc. ACM SIGMOD International Conference on Management of Data, 2003, pp. 76–85. https://doi.org/10.1145/872757.872770
2. R. M. Karp and M. O. Rabin, “Efficient randomized pattern-matching algorithms,” IBM Journal of Research and Development, vol. 31, no. 2, pp. 249–260, 1987.
3. U. Manber and G. Myers, “Suffix arrays: A new method for on-line string searches,” SIAM Journal on Computing, vol. 22, no. 5, pp. 935–948, 1993.
4. T. Kasai, G. Lee, H. Arimura, S. Arikawa and K. Park, “Linear-time longest-common-prefix computation in suffix arrays and its applications,” in Combinatorial Pattern Matching (CPM 2001), LNCS 2089, pp. 181–192, 2001.
5. A. Aiken, “MOSS: A system for detecting software similarity,” Stanford University. https://theory.stanford.edu/~aiken/moss/
6. L. Prechelt, G. Malpohl and M. Philippsen, “Finding plagiarisms among a set of programs with JPlag,” Journal of Universal Computer Science, vol. 8, no. 11, pp. 1016–1038, 2002.
7. S. Brin, J. Davis and H. Garcia-Molina, “Copy detection mechanisms for digital documents,” in Proc. ACM SIGMOD International Conference on Management of Data, 1995, pp. 398–409.
8. A. Z. Broder, “On the resemblance and containment of documents,” in Proc. Compression and Complexity of Sequences, IEEE, 1997, pp. 21–29.
9. M. Joy and M. Luck, “Plagiarism in programming assignments,” IEEE Transactions on Education, vol. 42, no. 2, pp. 129–133, 1999.
10. Dolos: Source code plagiarism detection, Ghent University. https://dolos.ugent.be/
11. Turnitin: Similarity checking and originality reports. https://www.turnitin.com/
12. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.